TRAVAIL PRATIQUE 2

CampusRate

Conception d’une API REST professionnelle

420-514

Collecte et interprétation des données

Techniques de l’informatique

Automne 2026

Informations générales

Élément	Information
Travail	TP2 : CampusRate
Réalisation	**Individuelle**
Durée	**Deux semaines (voir la date de remise sur Léa)**
Remise	Sur la boite de remise (sur Léa)
Technologies	NestJS, TypeScript, MongoDB, Mongoose, Swagger/OpenAPI, Postman

1. Mandat

Au TP1, CampusRate a été conçu comme une API REST permettant de consulter des endroits ou services du campus et de publier des appréciations. Dans ce deuxième travail, on doit faire évoluer cette API existante sans la reconcevoir à partir de zéro.

Le TP2 vise à :

1. remplacer la persistance JSON par MongoDB;
2. introduire une validation systématique des données reçues;
3. renforcer la sécurité de l'API et de sa configuration;
4. conserver autant que possible le contrat HTTP existant;
5. mettre à jour la documentation et vérifier le nouveau comportement.

2. Principe général

Le passage de JSON à MongoDB est un changement interne à l'application. Il ne justifie pas, à lui seul, la création d'une nouvelle version de l'API. Les URI, méthodes HTTP, formats de réponse et principaux comportements du TP1 doivent donc demeurer compatibles, sauf lorsqu'une modification est nécessaire et justifiée.

3. Partie obligatoire (70 points)

L'ensemble de cette partie doit être fonctionnel avant d'entreprendre les options.

3.1 Persistance avec MongoDB

La persistance JSON du TP1 doit être remplacée par MongoDB. Exigences :

- connecter l'application à MongoDB avec Mongoose;
- fournir l'URI MongoDB à partir de la configuration;
- créer les schémas Mongoose nécessaires aux deux concepts CampusRate;
- conserver les relations et règles métier pertinentes;

- remplacer les lectures et écritures dans les fichiers JSON par des opérations MongoDB;
- conserver une séparation claire entre contrôleurs, services et accès aux données;
- conserver autant que possible le contrat HTTP du TP1.

Le fichier JSON du TP1 ne doit plus être utilisé comme stockage principal.
Configuration minimale : `MONGO_URI=`

Le véritable fichier .env ne doit pas être versionné. Le fichier .env.example doit documenter les variables nécessaires sans contenir de secret réel.

Élément	Points
Connexion et configuration MongoDB	8
Schémas Mongoose	8
Remplacement de la persistance JSON	14
Maintien du contrat existant	5
Sous-total	35

4. Validation des données

Les données provenant d'un client doivent être considérées comme non fiables.

4.1 DTO

On doit utiliser des DTO adaptés au minimum pour la création et la modification.

Les validations doivent être pertinentes par rapport aux propriétés de CampusRate.

Il n'est pas attendu d'ajouter des décorateurs sans justification uniquement pour multiplier les validations.

4.2 Validation globale

L'application doit utiliser un ValidationPipe global.

- valider les propriétés reçues;
- supprimer ou refuser les propriétés non autorisées;
- empêcher le client d'imposer des propriétés contrôlées par le serveur.

4.3 Champs contrôlés par le serveur

Un client ne doit pas pouvoir imposer ou modifier directement des propriétés telles que : id, createdAt, updatedAt, averageRating, reviewCount

On doit également éviter de transmettre directement des objets fournis par le client à MongoDB.

Élément	Points
DTO de création et modification	8
Validation globale	6
Protection des champs et des entrées MongoDB	6
Sous-total	20

5. Sécurité HTTP de base

La sécurité d'une API ne se limite pas à l'authentification. Dans le socle obligatoire, on doit appliquer deux mécanismes de durcissement HTTP.

5.1 CORS

CORS doit être configuré explicitement. On doit éviter une configuration ouverte sans justification. La configuration doit permettre d'identifier les origines autorisées pour l'environnement utilisé.
On doit être capable d'expliquer que CORS limite certaines requêtes effectuées par les navigateurs, mais ne constitue pas une authentification de l'API.

5.2 En-têtes de sécurité avec Helmet

L'application doit utiliser Helmet afin d'ajouter des en-têtes HTTP de sécurité pertinents. Il n'est pas demandé de configurer individuellement chaque en-tête, mais on doit pouvoir expliquer brièvement le rôle général de Helmet.

Élément	Points
Configuration CORS	4
Helmet	3
Sous-total	7

6. Documentation et vérification

Swagger/OpenAPI doit être mis à jour pour correspondre au comportement réel de l'API après le passage à MongoDB et l'ajout des validations.

- propriétés obligatoires;
- contraintes principales;
- réponses d'erreur pertinentes;
- comportements modifiés.

La technologie de persistance MongoDB n'a pas à apparaître dans les URI du contrat REST.

6.1 Vérification manuelle

On doit démontrer, avec un enregistrement vidéo, au minimum les scénarios suivants :

1. création d'une ressource valide;
2. consultation d'une ressource persistée dans MongoDB;
3. persistance après redémarrage de l'API;
4. création avec une propriété obligatoire absente;
5. valeur invalide pour une propriété contrôlée;
6. propriété inconnue ou interdite envoyée par le client;
7. modification valide;
8. tentative de modification d'un champ contrôlé par le serveur.

Swagger UI ou Postman peuvent être utilisés.

Élément	Points
Swagger/OpenAPI	4
Vérifications manuelles	4
Sous-total	8

7. Résumé du socle obligatoire

Bloc	Points
MongoDB et Mongoose	35
Validation	20
Sécurité HTTP	7
Documentation et vérification	8
Total obligatoire	70

8. Fonctionnalités optionnelles (maximum 30 points)

Il n'est pas nécessaire de réaliser toutes les options. On choisit les fonctionnalités que l'on souhaite approfondir. Le maximum comptabilisé est de 30 points. Vous pouvez atteindre les 40 points pour compenser le score des fonctionnalités optionnelles si les 30 points ne sont pas atteints (pour des options qui ne sont pas entièrement fonctionnelles par exemple).

Une option donne des points seulement si elle est fonctionnelle, pertinente, intégrée proprement, documentée et démontrable.

8.1 MongoDB avancé

Code	Option	Points
M1	Ajouter un index pertinent, un index unique et gérer correctement une duplication	+6
M2	Réaliser une agrégation MongoDB pertinente	+4
M3	Justifier le choix entre référence et imbrication	+2

8.2 Sécurité HTTP avancée

Code	Option	Points
H1	Ajouter du rate limiting	+4
H2	Limiter la taille du corps HTTP et gérer 413	+2
H3	Vérifier le Content-Type et gérer 415	+2
H4	Restreindre davantage CORS selon l'environnement	+2
H5	Restreindre explicitement les méthodes HTTP autorisées	+1

8.3 HTTPS / TLS

Code	Option	Points
T1	Expliquer correctement le rôle de HTTPS dans le README	+1
T2	Faire fonctionner CampusRate localement en HTTPS	+5
T3	Générer et utiliser un certificat local autosigné	+2
T4	Mettre en place une redirection HTTP → HTTPS	+2

Une même réalisation ne peut donner des points plusieurs fois pour exactement le même travail.

8.4 Protection supplémentaire contre les entrées dangereuses

Code	Option	Points
P1	Utiliser une allowlist explicite de filtres	+2
P2	Sélection explicite des propriétés modifiables	+2
P3	Ajouter une protection supplémentaire contre les injections NoSQL	+3

8.5 Journalisation des opérations : INFO

Option L1 : +3 points

Ajouter une journalisation des opérations normales de l'API, par exemple : timestamp, method, path, statusCode, duration, requestId (facultatif).

Les informations sensibles ne doivent pas apparaître dans les logs.

8.6 Journalisation des erreurs : ERROR

Option L2 : +3 points

Ajouter une journalisation distincte pour les erreurs applicatives ou techniques, avec suffisamment de contexte pour faciliter le diagnostic sans exposer inutilement des secrets, mots de passe, URI MongoDB complète, jetons ou données confidentielles.

La stack trace interne peut être utile dans les logs techniques, mais elle ne doit pas être retournée telle quelle au client.

Les options L1 et L2 sont indépendantes.

8.7 Authentification avec JWT

Option A1 : +5 points.

- permettre une authentification;
- émettre un JWT;
- définir une expiration;
- vérifier correctement le jeton;
- protéger au moins une opération;
- documenter la sécurité Bearer dans Swagger.

Le refresh token n'est pas demandé.

8.8 Autorisation avec RBAC

Option A2 : +5 points.

Ajouter une autorisation basée sur au moins deux rôles : USER, ADMIN

Exemple possible pour CampusRate :

```
USER
- publier ou gérer ses propres appréciations

ADMIN
- gérer les endroits
- modérer les appréciations
```

RBAC nécessite une méthode fiable permettant d'identifier l'utilisateur. JWT n'est pas la seule possibilité, mais une authentification fonctionnelle doit exister.

9. Exemples de choix

Profil	Options possibles	Total indicatif
Sécurité HTTP	H1 + H2 + H3	+8
Sécurité MongoDB	M1 + M2	+10
Observabilité	L1 + L2	+6
Authentification	A1 + A2	+10
Total des points des fonctionnalités optionnelles		+34 (maximum de 30 points comptabilisés)

10. Gestion des secrets

Les éléments suivants ne doivent jamais être publiés dans le dépôt Git :

- .env;
- mot de passe MongoDB;
- jeton;
- clé privée;
- secret JWT;
- chaîne de connexion contenant des informations sensibles.

Le dépôt doit fournir un .env.example avec les noms des variables nécessaires, mais sans valeurs sensibles.

11. README

Le README existant doit être mis à jour afin de permettre à une personne externe au projet de :

1. installer les dépendances;
2. configurer MongoDB;
3. créer son fichier .env;
4. démarrer l'application;
5. accéder à Swagger;
6. connaître les principales validations;
7. connaître les fonctionnalités optionnelles réellement implantées;
8. comprendre les limitations connues.

Une option non terminée ne doit pas être présentée comme fonctionnelle.

12. Utilisation de l'intelligence artificielle

L'utilisation d'outils d'intelligence artificielle générative demeure permise selon les règles déjà appliquées au TP1.

- expliquer un message d'erreur;
- proposer une structure de README;
- expliquer une API de Mongoose;
- comparer deux stratégies;
- commenter une solution déjà produite;
- aider à diagnostiquer un problème.

L'étudiant demeure responsable du code remis, de sa compréhension, de son fonctionnement, des choix de sécurité et de l'exactitude de la documentation. Il doit être capable d'expliquer le code conservé dans son projet.

La déclaration de l'utilisation de l'IA doit être réalisée à l'aide de IAGraphie, selon les consignes du Cégep.

13. Remise

La remise comprend :

1. le dépôt GitHub;
2. le code source;
3. La vidéo de démonstration des fonctionnalités implémentées pour ce TP (lien de la vidéo);
4. .env.example;
5. le README à jour;
6. la documentation Swagger/OpenAPI;
7. les scénarios ou la collection Postman utilisés pour la vérification;
8. l'identification claire des options réalisées;
9. la déclaration d'intégrité/IAGraphie demandée.

Avant la remise :

```
npm ci
npm run lint
npm run build
```

doivent s'exécuter correctement.

14. Grille d'évaluation

Critère obligatoire	Points
Connexion/configuration MongoDB	8
Modélisation Mongoose	8
Migration de la persistance	14
Maintien du contrat HTTP	5
DTO	8
Validation globale	6
Protection des entrées et champs serveur	6
CORS	4
Helmet	3
Swagger/OpenAPI	4
Vérifications manuelles	4
Sous-total obligatoire	70

Options : maximum 30 points comptabilisés, même si la somme des options réalisées dépasse 30.

Note finale = partie obligatoire + options retenues, maximum 100 points.